x86-64 Assembly Programming Problem Set

Practice Programming problems

1 Problem #1

Conditional Factorial. *(100 pts)*

Write a working x86-64 assembly language program that computes the conditional factorial of a given non-negative integer n.

Conditions:

1. If n is **odd**, the output is the product of all the odd numbers within its factorial (up to n).
2. If n is **even**, the output is the product of all the even numbers within its factorial (up to n).

Sample Computation:

- For an odd number $n = 7$: $7!! = 7 \times 5 \times 3 \times 1 = 105$.
- For an even number $n = 8$: $8!! = 8 \times 6 \times 4 \times 2 = 384$.

Example:

```
Input:
14

Output:
645120
```

2 Problem #2

XOR Fibonacci Encryption/Decryption. *(100 pts)*

Develop an x86-64 program that can perform XOR Fibonacci Encryption/Decryption.

In an XOR encryption system, each character in a string is XOR-ed with a key. The resulting encrypted string can simply be decrypted by performing the XOR operation with the same key.

For example, given the input text `"Hello"` and key (in hex) `1A 2B 3C 4D 5E`:

```
Encryption process
Input Text = 48 65 6C 6C 6F ("Hello" in ASCII)
Key        = 1A 2B 3C 4D 5E
Encrypted  = 52 4E 50 21 31 ("RNP!1" in ASCII)

Decryption process
Input Text = 52 4E 50 21 31 ("RNP!1" in ASCII)
Key        = 1A 2B 3C 4D 5E
Decrypted  = 48 65 6C 6C 6F ("Hello" in ASCII)
```

Observe how the key is given, and how it must remain consistent between encryption and decryption processes.

In an XOR Fibonacci encryption, the key depends on the Fibonacci sequence. The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding numbers. Typically, the sequence begins with 0 and 1. However, in this case, let the user decide on the two starting numbers, n_0 and n_1, both of which are to be declared in memory and limited to the size of a byte. Throughout the sequence, let each element in the sequence be truncated to 8 bits. Lastly, the length of the sequence to be generated should match the length of the input text to be encrypted/decrypted (not including the null terminating character).

For example, let n_0 and n_1 be 74 and 108, respectively (in decimal), where the input text is `"LBYARCH"`. Since the input text is seven characters long, the key is `4A 6C B6 22 D8 FA D2` (in hex) since:

Sequence Number	Sequence Value (in DEC)	Sequence Value (in HEX)
1	74	4A
2	108	6C
3	182	B6
4	34	22
5	216	D8
6	250	FA
7	210	D2

To summarize the requirements of this program, let the user define the following in memory: input string, n_0, and n_1. You may assume that the length of the input string is limited to 20 characters. Let the program provide an output following the format below:

1. Input Text (in ASCII)
2. Input Text (in HEX, each character separated by a space)
3. Key (in HEX, each character separated by a space)

4. Encrypted Text (in HEX, each character separated by a space)
5. Encrypted Text (in ASCII)

Example:

```
Input (in memory):
Text = "Assembly"
n0 = 25
n1 = 138

Output:
Assembly
41 73 73 65 6D 62 6C 79
19 8A A3 2D D0 FD CD CA
58 F9 D0 48 BD 9F A1 B3
XùÐHi3
```

3 Problem #3

Integer Sequence Generator. *(100 pts)*

Write a working x86-64 assembly language to generate a sequence of integer numbers based on a positive integer input. Required to use functional x86-64 register.

Input: Prompt the user for input. Input should be a positive integer (not a string).

- **Error checking:** Invalid input. If an error exists, output an appropriate error message and input it again.

Process:

- If the integer is divisible by three → divide it by three.
- If the integer is not divisible by three but is even → divide it by two.
- If the integer is not divisible by three but is odd → multiply it by two and subtract one.
- Repeat the process until the number reaches one or if it reaches ten sequences (inclusive).

Output: The sequence of integers. Prompt the user whether the program will be executed again.

Example:

```
Input: 10
Sequence: 10,5,9,3,1
Do you want to continue (Y/N)? Y

Input: -12
Error: negative number input
Do you want to continue (Y/N)? Y

Input Number: XYZ
```

```
Error: non-numeric input
Do you want to continue (Y/N)? Y

Input: 99
Sequence: 99,33,11,21,7,13,25,49,97,193
Do you want to continue (Y/N)? N
```

4 Problem #4

Capitalize First Letter of Each Word. *(15 pts)*

Write an x86-64 program that capitalizes the first letter in each word of a given sentence. Words in a given sentence are separated by the space character (ASCII: `0x20`).

Hint: The ASCII values of letters in lowercase have a difference of `0x20` from the ASCII values of letters in uppercase (i.e., `'a' = 0x61` and `'A' = 0x41`). If the first letter of the word, or the entire word, is already in uppercase, then it does not need to be converted.

Input [Initialized/hardcoded in memory]: Null terminated string with a maximum length of 200 characters.

Output [Message dialog box]: String where the first character in each word is capitalized.

Example #1:

```
var db "the quick brown fox jumps over the lazy dog.",0

Screen Output: The Quick Brown Fox Jumps Over The Lazy Dog.
```

Example #2:

```
var db "I am having fun in LBYARCH",0

Screen Output: I Am Having Fun In LBYARCH
```

5 Problem #5

Join 2 Datasets. *(15 pts)*

Create an x86 program that joins 2 datasets $A = [a_1, a_2, a_3, \dots]$ and $B = [b_1, b_2, b_3, \dots]$ where each element of A is combined with every element of B:

$$AB = [a_1, b_1, b_2, b_3, \dots, a_2, b_1, b_2, b_3, \dots]$$

Provide the length of the dataset followed by its element as the input. Assume that the maximum number of elements in a dataset is 100.

Example:

```
A has [7, 8]
B has [1, 2, 3]

Inputs:
2 7 8 3 1 2 3

Output:
7 1 2 3 8 1 2 3
```